MindMatrix VTU Internship Program

Project Report

Project Title #25

Android App Development using GenAI Skill-Exchange

(Self-Employment)

Project Number	25
Platform	Android (Kotlin + XML Layouts)
Domain	Rural Economy / Self-Employment / Social Commerce
Category	Android Application with Firebase & GenAI Integration
Report Type	Internship / Academic Project Report

Prepared By

Arshid Ahmad Malik
USN: 1MJ23EC401
E-Mail: 1mj23ec401@mvjce.edu.in
MVJ College of Engineering
Department of Electronics and Communication Engineering
2025 – 2026

1. Introduction

Skill-Exchange is an Android application developed as part of the MindMatrix VTU Internship Program under Project Title #25. The application addresses a critical socio-economic challenge faced by rural and semi-urban communities in India: village technicians such as plumbers, electricians, carpenters, and masons frequently need each other's assistance but lack the cash to pay for services. Skill-Exchange provides a structured digital platform that enables these artisans to trade their skills through a Barter Economy model — exchanging hours of work rather than money.

The application is built on Android using Kotlin and XML-based layouts. It leverages Firebase Firestore as a real-time "Skill Board" database, integrates a GenAI-powered matching engine to suggest compatible skill swaps, and incorporates a Trust Score system that quantifies community reliability through successful swap completions.

Skill-Exchange targets the underserved informal labour economy in rural India — a population that historically relies on verbal agreements, personal contacts, and goodwill to access technical services. By digitising the barter process and assigning measurable value to hours of work (1 hour = 1 Skill Point), the app creates a self-reliant, cashless ecosystem of skilled community workers.

2. Problem Statement

India's rural informal economy is home to millions of skilled technicians who operate independently — plumbers, electricians, welders, carpenters, roofers, and more. These workers frequently need specialised assistance from peers to complete jobs that require multiple skill sets. However, several structural barriers prevent effective collaboration:

Challenge	Description
Cash Dependency	Technicians cannot afford to pay each other in cash for assistance, even when the service is urgently required, leading to deferred or incomplete work.
Lack of Visibility	Workers do not know who in their community possesses what skills. There is no structured directory or discovery platform for rural artisans.
No Accountability	Verbal barter agreements lack documentation, leading to disputes, reneging, and erosion of community trust over time.
Skill Undervaluation	Traditional manual trades are socially undervalued, with no formal mechanism to recognise or reward the expertise of rural craftspeople.
Isolation	Workers depend on outside contractors and urban service providers for specialised skills, increasing costs and reducing local economic selfreliance.

3. Project Vision

Skill-Exchange envisions a world where every rural artisan — regardless of education, income, or location — can access the skills they need and contribute the skills they have, without the barrier of cash. The platform aspires to create a self-sustaining "Barter Economy" among rural technicians, powered by digital trust, AI-driven matching, and community accountability.

The project is built on four foundational pillars:

- **Community Self-Reliance –** Reducing rural dependency on expensive outside contractors by enabling local technicians to collectively address each other's needs.
- **Skill Valorisation –** Assigning formal, measurable value to traditional manual trades through the Skill Point system (1 hour = 1 point), creating a universal currency of time.
- **Digital Trust –** Building accountable relationships through a Trust Score mechanism that rewards completed swaps and penalises defaults, making reliability visible and incentivised.
- **Social Capital –** Fostering cooperation, goodwill, and mutual dependency among community workers, strengthening the social fabric of rural settlements.

4. Scope of the Application

4.1 Target Users

- Village plumbers, electricians, carpenters, welders, masons, and roofers in rural and semi-urban India
- Self-employed daily-wage skilled workers who lack formal employment or access to organised labour markets
- Community artisans seeking to build a reputation and expand their network of collaborators
- Small construction or repair project owners who cannot afford to hire multiple tradespeople

4.2 Platform

- Android smartphones with minimum API Level 21 (Android 5.0 Lollipop and above)
- Optimised for low-end and mid-range devices common in rural India
- Requires internet connectivity for Firebase real-time sync and AI-powered swap suggestions

4.3 Key Operational Scenarios

- A carpenter listing their skills and posting a need for roofing assistance on the Skill Board
- An electrician browsing posted needs filterable by "Skill Required" to find relevant swap opportunities
- A plumber offering to fix an electrician's pipes in exchange for motor repair work
- Both parties confirming a completed swap, triggering Trust Score updates for both users
- A new artisan viewing the leaderboard to identify the most trusted and reliable community members

5. Objectives

The project is aligned with the following measurable objectives:

1. **Skill Profile Creation –** Enable every artisan to create a digital profile declaring their expertise (e.g., "I am an Expert Carpenter"), discoverable by other community members.
2. **Need Post System –** Allow users to post specific help requests (e.g., "Need help with leaking roof") filterable by skill category, enabling targeted matching.
3. **Swap Offer Mechanism –** Implement a structured swap proposal interface where users can offer their skills in exchange for the help they need, e.g., "I will do your woodwork in exchange for roof repair."
4. **Trust Score System –** Build a quantitative community reliability metric that increases only when both parties mutually confirm a completed swap, preventing gaming or unilateral score manipulation.
5. **Skill Point Economy –** Implement a time-based value system where 1 hour of skilled work = 1 Skill Point, creating a universal measure for equitable skill exchange.
6. **Real-time Skill Board –** Leverage Firebase Firestore to maintain a live, community-wide board of posted needs and available skills with instant updates.
7. **Community-Focused UI –** Design an accessible, friendly, and simple interface that is usable by workers with varying levels of digital literacy.

6. Features

6.1 Core Features

Feature	Description

Skill Profile	Each user creates a profile declaring their trade expertise (e.g., "Expert Carpenter", "Licensed Plumber"). The profile is publicly visible on the Skill Board and includes skill tags and current Trust Score.
Need Post	Users post specific help requests with skill category tags (e.g., "Roofing", "Electrical", "Plumbing"). Posts are filterable by Skill Required, making discovery efficient.
Swap Offer	In response to a Need Post, users submit a structured Swap Offer specifying what they will provide and what they need in return. The chat-like interface supports negotiation before agreement.
Skill Point System	Time-based value currency: 1 hour of skilled work = 1 Skill Point. Points are proposed during swap negotiation and recorded upon mutual confirmation of completion.
Trust Score	A numeric reputation score visible on every user's profile. The Trust Score increases only when both parties confirm swap completion, ensuring bilateral accountability.
Firebase Skill Board	All Need Posts, Skill Profiles, and Swap Offers are stored and synced in real time using Firebase Firestore, enabling community-wide visibility with zero latency.
Swap Chat Interface	A simple, WhatsApp-style chat interface allows two parties to negotiate swap terms, agree on Skill Points, and confirm completion, all within the app.
Skill Board Filter	Need Posts are filterable by Skill Required category (e.g., show only "Plumbing" needs), allowing workers to quickly identify relevant opportunities.
Community Leaderboard	A ranked leaderboard of users by Trust Score, showcasing the most reliable and active community members, incentivising participation and accountability.

6.2 Future Scope Features

- Rating & Review System – Allow users to leave qualitative feedback after each swap, enriching Trust Score with contextual detail.
- Voice-Based Posting – Enable illiterate or low-literacy users to post needs and offers using voice input with speech-to-text conversion.
- Multi-Language Support – Extend the interface to support Kannada, Hindi, Tamil, and Telugu in addition to English.
- Offline Queue – Allow Need Posts to be drafted offline and automatically submitted when connectivity is restored.

- Skill Certification Badge – Issue digital badges for users who complete 10+ verified swaps in a specific skill category, building portfolio credibility.

- Integration with Government Skill India Programs – Link verified skill profiles with national skilling databases for formal recognition.

7. Functional Requirements

The following functional requirements define the core behaviours the system must support:

FR-1: User Registration & Profile

- The system shall allow new users to register with name, phone number, and declared skill expertise.
- The system shall store profile data in Firebase Firestore with a unique user ID.

- Users shall be able to update their skill profile at any time.
- The Trust Score shall be initialised at 0 for all new users and displayed prominently on the profile.

FR-2: Need Post Management

- The system shall allow users to create a Need Post specifying the type of help required and the skill category.
- The system shall validate that the Need Post contains a description and at least one skill tag before submission.
- Need Posts shall be visible to all registered community members on the Skill Board in real time.
- The system shall support filtering of Need Posts by Skill Required category.
- Users shall be able to delete their own Need Posts after a swap is completed or the need is resolved.

FR-3: Swap Offer

- Any user may submit a Swap Offer in response to any open Need Post.
- The Swap Offer shall include a description of the service being offered and the Skill Points proposed.
- The Need Post owner shall receive a notification upon receipt of a new Swap Offer.
- The system shall support negotiation via a chat-style interface between the two parties.

FR-4: Skill Point System

- The system shall maintain a Skill Point balance for each user, initialised at 0.
- Skill Points are proposed during swap negotiation (1 hour = 1 point) and recorded upon bilateral confirmation.
- The Skill Point ledger shall be viewable by the owning user from their profile.

FR-5: Trust Score

- The Trust Score shall increment only after both parties explicitly confirm swap completion within the app.
- A swap confirmed by only one party shall not alter either party's Trust Score.
- The Trust Score shall be visible on each user's public profile and on the community leaderboard.

FR-6: Community Leaderboard

- The system shall maintain a leaderboard ranking all users by Trust Score in descending order.
- The leaderboard shall update in real time as Trust Scores change following swap confirmations.
- The top 10 users shall be highlighted with a badge indicator.

8. Non-Functional Requirements

Category	Requirement	Acceptance Criterion
Performance	Skill Board Load Time	Skill Board must fully render within 1200 ms of login on a mid-range Android device.
Performance	Swap Offer Submission	A Swap Offer must be submitted and visible to the Need Post owner within 2 seconds.
Usability	UI Simplicity	Any user must be able to post a Need and submit a Swap Offer within 4 taps.

Usability	Filter Responsiveness	Skill Board filter must return filtered results within 800 ms of category selection.
Reliability	Data Consistency	Trust Score updates must be atomic — no partial update must be visible to either party.
Reliability	Crash Rate	Application crash rate must not exceed 1% during standard interaction flows.
Security	Authentication	User accounts must be secured via Firebase Authentication (phone OTP or email).
Compatibility	Android Versions	Application must support Android 5.0 (API 21) through Android 14 (API 34).
Accessibility	Touch Targets	All interactive elements must meet 48x48 dp minimum touch target size.

9. System Architecture

Skill-Exchange follows a client-cloud architecture. The Android client handles all UI rendering, user interaction, and local state management. Firebase Firestore serves as the real-time backend database, and Firebase Authentication manages user identity. A GenAI integration layer provides intelligent swap match suggestions.

9.1 Technology Stack

Layer	Technology	Purpose
Language	Kotlin	Primary development language; concise, nullsafe, fully interoperable with Android SDK.
UI Framework	XML Layouts + AppCompat	View-based UI with RecyclerView, CardView, and Material Design components for broad device compatibility.
Backend Database	Firebase Firestore	Real-time NoSQL cloud database acting as the Skill Board; stores profiles, Need Posts, Swap Offers, and Trust Scores.
Authentication	Firebase Authentication	Handles user registration and login via phone number OTP or email, ensuring secure identity management.
AI Integration	GenAI API (Gemini)	Analyses posted needs and available skill profiles to suggest optimal swap matches and recommend compatible partners.
Local Storage	Room Database	Caches Skill Board data locally for faster load times and partial offline browsability.
Push Notifications	Firebase Cloud Messaging	Delivers real-time notifications for new Swap Offers, swap confirmations, and Trust Score updates.
Build System	Gradle (Groovy DSL)	Dependency management, build configuration, and APK generation.
Min. SDK	API Level 21	Targets approximately 98% of active Android devices, including affordable smartphones prevalent in rural India.

9.2 Architectural Overview

The application follows an MVVM (Model-View-ViewModel) pattern adapted for Firebase-backed Android development:

- Model Layer – Firebase Firestore collections (users, needPosts, swapOffers, swapHistory) and a local Room cache. Data classes (UserProfile, NeedPost, SwapOffer, SwapHistory) represent document schemas.

- View Layer – XML layout files define the UI structure (activity_main.xml, activity_skillboard.xml, activity_profile.xml, activity_swap_chat.xml, activity_leaderboard.xml). Custom drawables provide visual styling.
- ViewModel Layer – SkillBoardViewModel, ProfileViewModel, and SwapViewModel expose LiveData streams backed by Firestore snapshot listeners. ViewModels survive configuration changes and manage Firestore subscriptions.
- Repository Layer – SkillBoardRepository and UserRepository abstract Firestore operations, Room caching logic, and GenAI API calls from ViewModels.
- Background Services – Firebase Cloud Messaging handles push notification delivery. WorkManager periodically syncs the local Room cache with Firestore for offline resilience.

9.3 Database Design (Firestore Collections)

Collection	Key Fields	Description
users	userId, name, skills, trustScore, skillPoints	Stores each user's profile including declared skills, Trust Score, and accumulated Skill Points.
needPosts	postId, userId, description, skillRequired, timestamp, status	All active help requests posted to the Skill Board, tagged by skill category and open/closed status.
swapOffers	offerId, postId, offererId, description, skillPoints, status	Swap proposals submitted in response to Need Posts. Status tracks negotiation → agreed → completed.
swapHistory	swapId, party1Id, party2Id, pointsExchanged, confirmedBy, timestamp	Immutable record of all completed and confirmed swaps, used for Trust Score calculation and audit.
chatMessages	messageId, swapOfferId, senderId, text, timestamp	Chat messages exchanged between two parties during swap negotiation, scoped to a specific Swap Offer.

10. User Flow

The application is designed to minimise friction in high-frequency use scenarios. The complete user journey from registration to a confirmed swap is described below:

#	Screen	User Action & System Response
1	**Registration / Login**	New user registers with name, phone, and skill expertise. OTP verified via Firebase Auth. Existing users log in directly.
2	**Skill Board (Home)**	User sees the live Skill Board showing all open Need Posts. Posts can be filtered by Skill Required. The user's Trust Score is visible in the top navigation bar.
3	**Post a Need**	User taps "Post Need", describes the help required, selects skill tags, and submits. The post appears on the community Skill Board in real time.
4	**Browse & Filter**	Another user browses the Skill Board and filters by skill category. They identify a relevant Need Post and tap to view full details.
5	**Submit Swap Offer**	The browsing user submits a Swap Offer: "I will do your woodwork in exchange for roof repair – 3 Skill Points." The Need Post owner receives a push notification.
6	**Swap Chat & Negotiation**	Both parties enter the swap chat interface, negotiate terms, agree on Skill Points, and mark the swap as "Agreed".
7	**Swap Execution**	The physical exchange of skills occurs offline. Both parties record that they have fulfilled their commitment within the app.
8	**Bilateral Confirmation**	Both parties independently confirm swap completion. Once both confirmations are received, the Trust Score for each party increases and Skill Points are recorded.
9	**Leaderboard Update**	The community leaderboard updates in real time, reflecting the new Trust Scores of both users.
10	**Profile View**	Any community member can view a user's public profile showing their skills, Trust Score, Skill Points, and swap history count.

11. Technical Implementation

11.1 Firebase Firestore as Skill Board

The Skill Board is implemented as a real-time Firestore collection (needPosts). Firestore snapshot listeners (addSnapshotListener) push updates to the client instantly whenever a new Need Post is created, updated, or deleted. The SkillBoardViewModel exposes a LiveData<List<NeedPost>> stream backed by this listener, ensuring the UI always reflects the latest community state without manual refresh.

Need Posts are stored as Firestore documents with indexed fields for skillRequired (enabling server-side filtering), timestamp (for recency ordering), and status (open/closed). Firestore compound queries combine skill tag filtering with timestamp ordering to deliver filtered Skill Board views efficiently.

11.2 Trust Score System

The Trust Score mechanism is implemented as a Firestore Cloud Function triggered on writes to the swapHistory collection. When a swap document reaches a confirmedBy count of 2 (both parties have confirmed), the Cloud Function atomically increments the trustScore field in both user documents using Firestore transactions, guaranteeing that partial updates cannot produce inconsistent states.

The design decision to use server-side Cloud Functions for Trust Score updates — rather than client-side writes — prevents malicious or accidental client manipulation of reputation scores, a critical integrity requirement for a community trust platform.

11.3 Skill Point System

Skill Points are proposed during swap negotiation in the chat interface and recorded in the swapHistory document upon bilateral confirmation. The system implements a simple ledger: each confirmed swap adds pointsExchanged to the recipient's skillPoints field via a Firestore transaction. A user's Skill Point balance is visible on their profile and represents their cumulative contribution to the community.

11.4 GenAI Integration for Swap Matching

The application integrates the Gemini GenAI API to provide AI-powered swap suggestions. When a user creates a Need Post, the app sends the post description and skill tags to the Gemini API along with the current list of active skill profiles. The AI analyses skill compatibility, availability signals, and historical swap patterns to recommend the top 3 most compatible swap partners, displayed as "Suggested Matches" on the Need Post detail screen.

11.5 Swap Chat Interface

The swap negotiation chat is implemented as a real-time Firestore-backed messaging system. Chat messages (chatMessages collection) are ordered by timestamp and delivered to both parties via snapshot listeners. The interface uses a RecyclerView with a custom ChatAdapter that differentiates sent and received messages by alignment and background colour. A confirmation action button at the bottom of the chat allows each party to mark their obligation as fulfilled, triggering the bilateral confirmation flow.

11.6 Need Post Filtering

The Skill Board filter is implemented as a Firestore compound query combining a where() clause on skillRequired with an orderBy() on timestamp. Filter selections are presented as a horizontal chip group at the top of the Skill Board screen. Selecting a chip fires a new Firestore query scoped to that skill category; selecting "All" removes the filter and returns the full open posts list. Room Database caches the last fetched result set for each filter combination, enabling rapid reloads from cache before the live Firestore result arrives.

12. Project File Structure

File / Directory	Purpose
app/build.gradle	App-level Gradle config: SDK versions, Firebase, Room, Gemini API, and kapt dependencies.
data/model/UserProfile.kt	Data class representing a user's Firestore document: name, skills, trustScore, skillPoints.
data/model/NeedPost.kt	Data class for a Skill Board Need Post: description, skillRequired, userId, timestamp, status.
data/model/SwapOffer.kt	Data class for a Swap Offer: offerId, postId, offererId, description, skillPoints, status.
data/repository/SkillBoardRepo.kt	Abstracts Firestore needPosts queries, Room cache writes, and GenAI match API calls.
data/repository/UserRepository.kt	Manages user profile reads/writes, Trust Score observing, and Skill Point balance queries.
ui/MainActivity.kt	Host activity with bottom navigation for Skill Board, Post Need, Profile, and Leaderboard tabs.
ui/SkillBoardFragment.kt	Displays real-time Need Posts list with filter chips. Hosts SkillBoardViewModel.
ui/PostNeedActivity.kt	Form for creating a new Need Post with description input and skill tag multi-select.
ui/SwapChatActivity.kt	Real-time chat interface for swap negotiation and bilateral completion confirmation.
ui/ProfileActivity.kt	Displays user profile: skills, Trust Score, Skill Points, and swap history count.
ui/LeaderboardActivity.kt	Community leaderboard ranked by Trust Score with realtime Firestore listener.
ui/LoginActivity.kt	Firebase Authentication UI for phone OTP or email-based registration and login.
viewmodel/SkillBoardViewModel.kt	Exposes LiveData<List<NeedPost>> backed by Firestore snapshot listeners and Room cache.
viewmodel/SwapViewModel.kt	Manages swap offer submission, chat message sending, and bilateral confirmation logic.
adapter/NeedPostAdapter.kt	RecyclerView adapter for rendering Need Post cards on the Skill Board.

adapter/ChatAdapter.kt	RecyclerView adapter for rendering sent/received chat messages in SwapChatActivity.
res/layout/*.xml	XML layout files for all activities and fragments: skill board, profile, swap chat, leaderboard.
google-services.json	Firebase project configuration file enabling Firestore, Auth, and FCM integration.

13. Permissions & Privacy

Permission	Required For	Notes
INTERNET	Firebase Firestore, Auth, FCM, GenAI API	Required for all core features. Core functionality requires active connectivity.
POST_NOTIFICATIONS	Swap Offer & confirmation alerts	Required on Android 13+. User prompted on first notification trigger.
RECEIVE_BOOT_COMPLETED	Restart FCM service after reboot	Ensures push notifications remain active after device restarts.

No sensitive personal data beyond name and phone number (required for authentication) is collected or stored externally. All data is stored in Firebase under the user's authenticated UID and governed by Firestore Security Rules that prevent one user from reading or writing another user's private data. Customer or community member data is never sold, shared with advertisers, or used for profiling beyond in-app swap matching.

14. Impact Goals

14.1 Reduction of Rural Dependency on External Labour

By enabling local technicians to find and exchange skills within their own community, SkillExchange directly reduces the need to hire expensive urban contractors for common rural maintenance and construction tasks. Villages that previously paid premium rates for outside plumbers or electricians can now resolve many needs internally, retaining economic value within the community.

14.2 Skill Valorisation

The Skill Point system assigns measurable, formal value to manual trades that are often socially undervalued. Earning Skill Points and Trust Score creates a publicly visible record of a worker's reliability and contribution, providing artisans with a digital reputation that can open doors to more formal employment and business opportunities.

14.3 Social Capital and Community Cooperation

Every successful skill swap recorded on the platform strengthens the bond between two community members and, by extension, the broader community network. The Trust Score leaderboard creates positive competitive incentives for reliability and generosity, building a culture of cooperation and mutual aid among workers who might otherwise operate in isolation.

14.4 Digital Inclusion

Skill-Exchange demonstrates that a meaningful, community-impacting digital platform can be delivered on affordable Android hardware with a simple, accessible interface. By targeting API 21 and designing for low-literacy users with icon-heavy, minimal-text UI components, the app extends the benefits of the digital economy to workers who have historically been excluded from it.

14.5 Scalability of Impact

The modular Firebase and MVVM architecture supports rapid expansion. Future versions can add voice-based posting for low-literacy users, integration with government Skill India databases for formal credential recognition, and a micro-lending layer where Skill Points serve as collateral for small community loans — without requiring a fundamental redesign of the existing codebase.

15. Success Criteria

The project will be considered successfully completed when the following criteria are met:

#	Success Criterion	Measurement	Priority
1	Skill Board loads and displays Need Posts within 1200 ms on a reference mid-range Android device.	Android Profiler measurement	Critical
2	A Need Post can be created and appears on the community Skill Board within 4 taps from login.	Manual UI walkthrough	Critical
3	Need Posts are correctly filterable by Skill Required with results returned within 800 ms.	Manual filter testing	Critical
4	Trust Score increments only after both parties independently confirm swap completion; singleparty confirmation produces no score change.	Manual security and logic test	Critical
5	Swap Offer notifications are delivered to the Need Post owner within 5 seconds of submission.	FCM delivery log verification	Critical
6	Leaderboard updates in real time and reflects correct Trust Score rankings after each confirmed swap.	Manual leaderboard validation	High
7	GenAI matching suggests at least 1 compatible swap partner for every Need Post where a matching profile exists.	AI response validation	High
8	App does not crash during complete walkthrough of all screens, flows, and edge cases.	Manual regression testing	High
9	The UI is navigable without explicit text instructions, validated by a test with a user unfamiliar with the app.	User observation session	Medium
10	All Firestore Security Rules prevent cross-user data access; no user can read or modify another user's private records.	Firebase Security Rules test suite	Medium

Conclusion

Skill-Exchange represents a targeted and socially purposeful application of Android development and Generative AI in the domain of rural self-employment and informal barter economies. By addressing a clearly defined problem — the inability of village technicians to access each other's skills due to cash constraints and lack of visibility — the project delivers both measurable practical value and technical merit.

The application's architecture prioritises real-time community interaction, mutual accountability, and AI-assisted discovery. Every design decision, from Firebase Firestore for live Skill Board updates, to the dual-confirmation Trust Score mechanism, to the Gemini-powered swap matching, is motivated by a single question: will this create a more self-reliant, cooperative, and economically resilient rural community?

The project also establishes a strong technical foundation for future expansion. The MVVM architecture with a Firebase backend and Room cache supports the addition of voice input, offline operation, multi-language UI, and government skill database integration without requiring a fundamental redesign. The Firestore Security Rules framework ensures that community trust data remains tamper-proof as the user base scales.

In conclusion, Skill-Exchange is not simply a technical deliverable — it is a small but meaningful step toward economic self-reliance, skill valorisation, and digital inclusion for the millions of rural artisans who form the backbone of India's informal economy.